Berkeley MSSE
MOLECULAR SCIENCE & SOFTWARE ENGINEERING

Lecture 13:

Clustering and Classification - Supervised

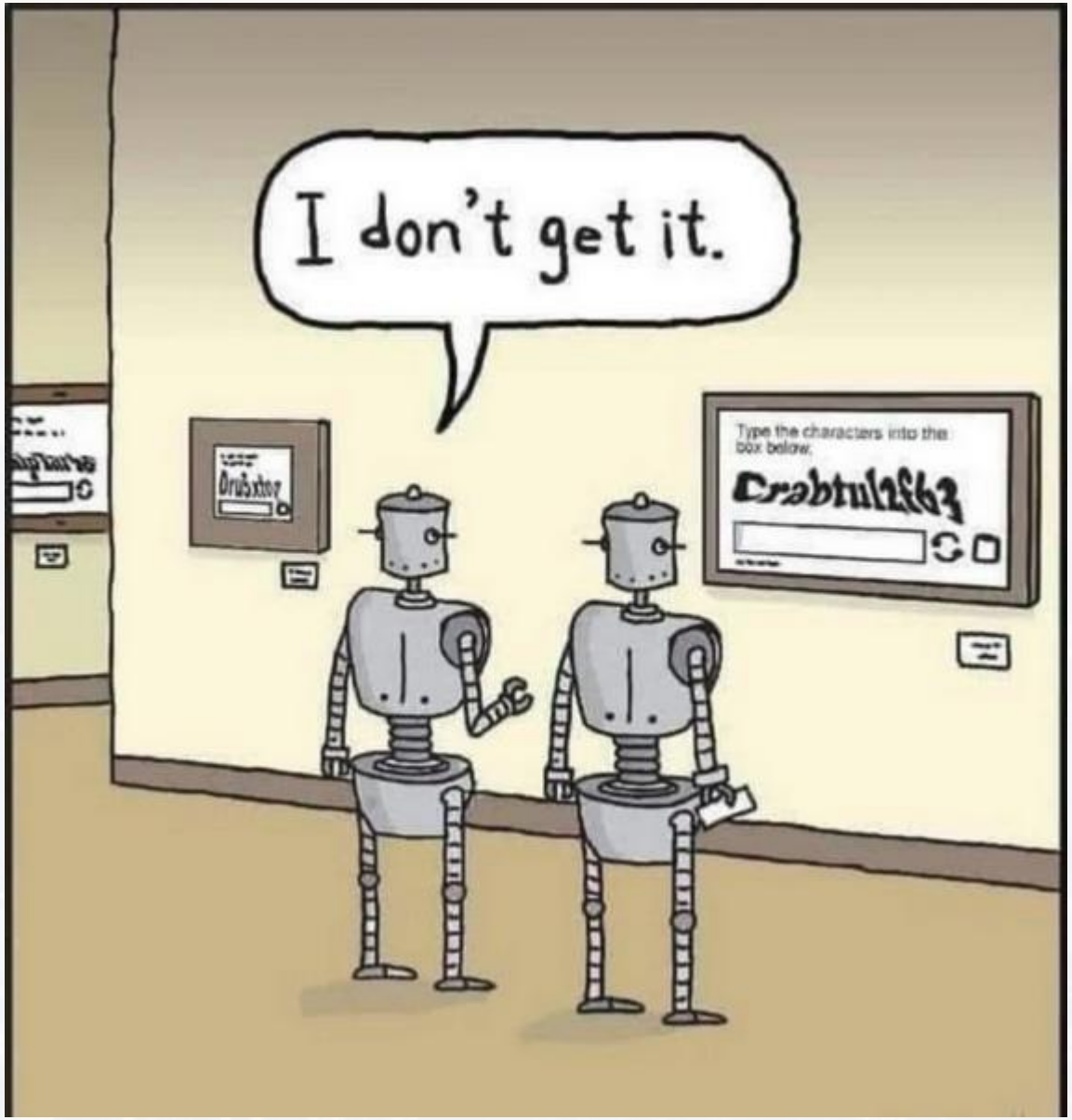
Markus Hohle

University of California, Berkeley

Data Science for Scientific Computing

MSSE 277A, 3 Units

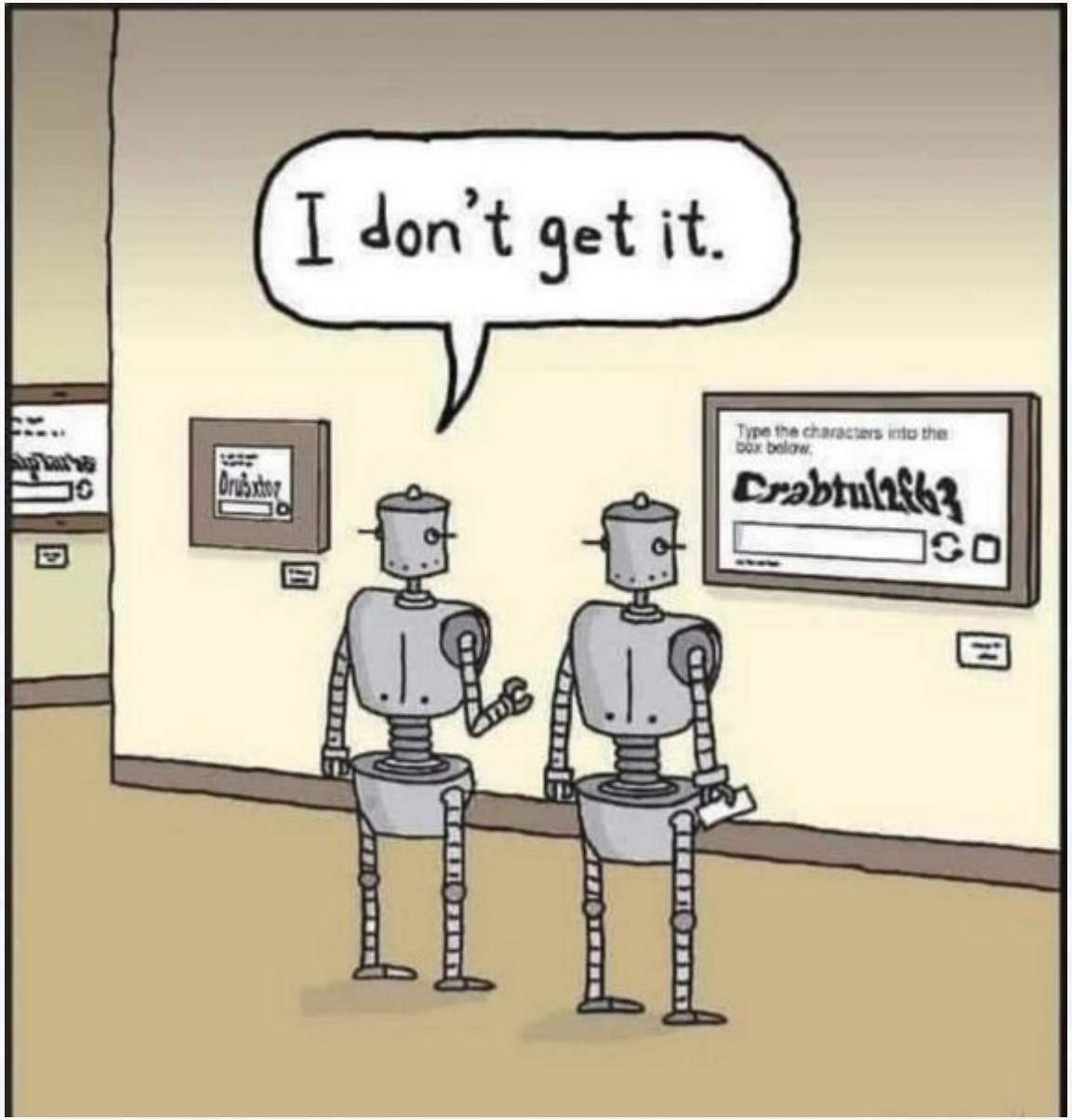
Clustering and Classification - Supervised



Outline

- **Overview**
- **Examples**
  - k-NN
  - Naïve Bayes
  - SVM

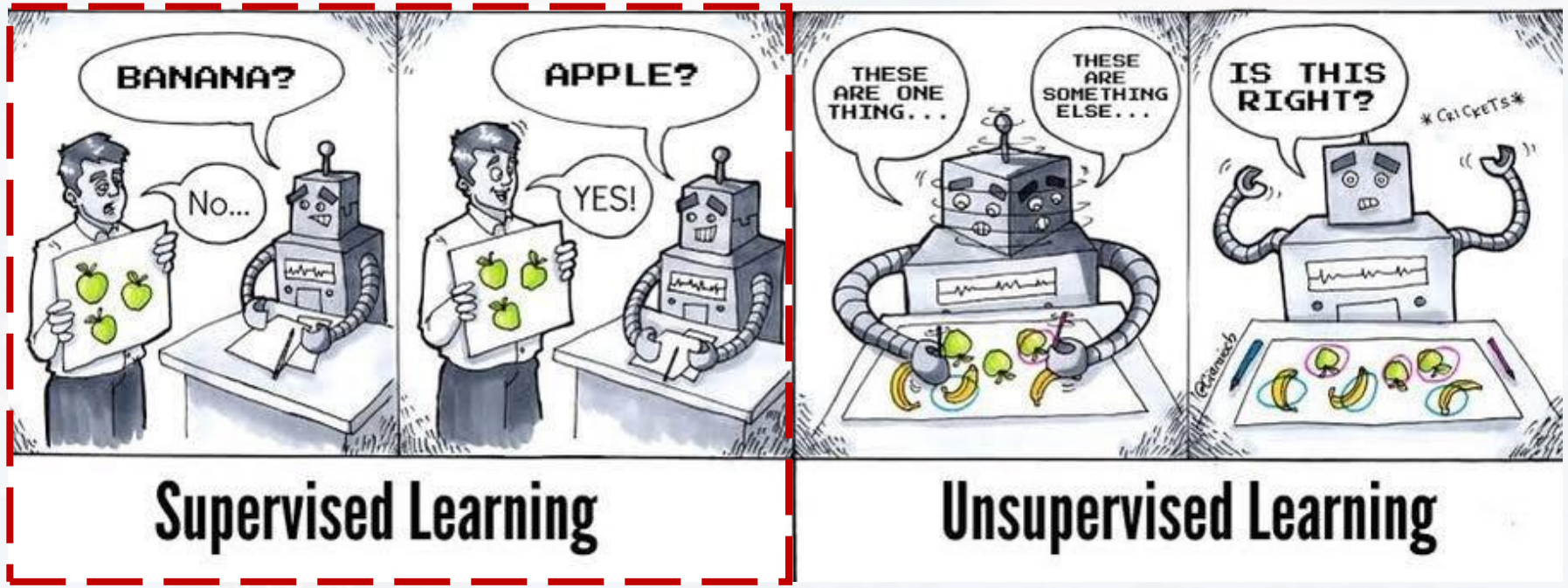
Clustering and Classification - Supervised



Outline

- **Overview**
- Examples
  - k-NN
  - Naïve Bayes
  - SVM

Clustering and Classification – Supervised



Clustering and Classification – Supervised

Overview

	supervised	unsupervised
regression	linear regression ✓ ANNs (soon)	
classification	logistic regression ✓ **K-nearest (today)** ✓ **SVM (today)** ✓ ANNs (soon) **Naïve Bayes (today)** ✓	GMM ✓ K-means ✓ Trees ✓ Hidden Markov Models

Clustering and Classification – Supervised

Workflow

supervised

1) creating the model:

```
my_model = library.method(argument1 = 'arg1', … )
```

2) training the model

```
out = my_model.fit(xtrain, ytrain)
```

3) evaluation

```
ypred = out.predict(xeval)
accur = (ypred == yeval).sum()/len(yeval)
```

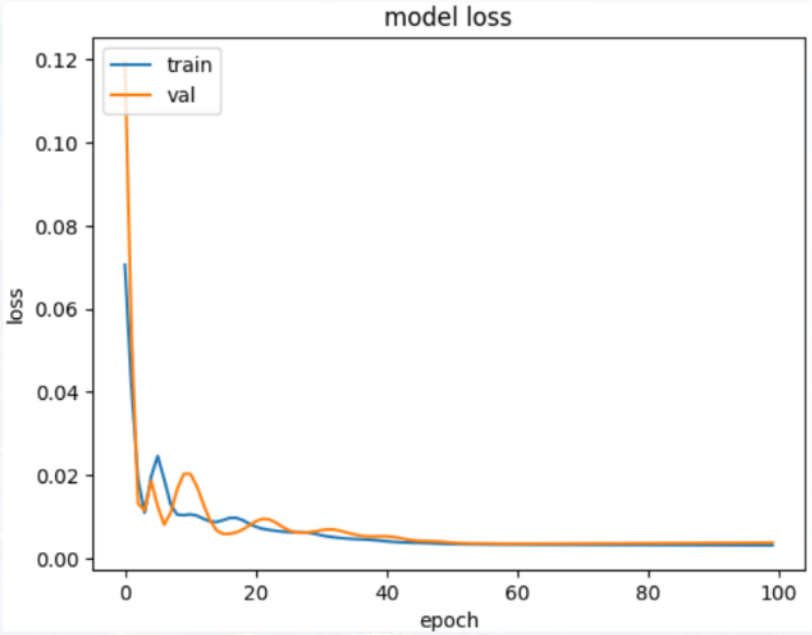
4) prediction (actual application)

```
ypred = out.predict(xnew)
```

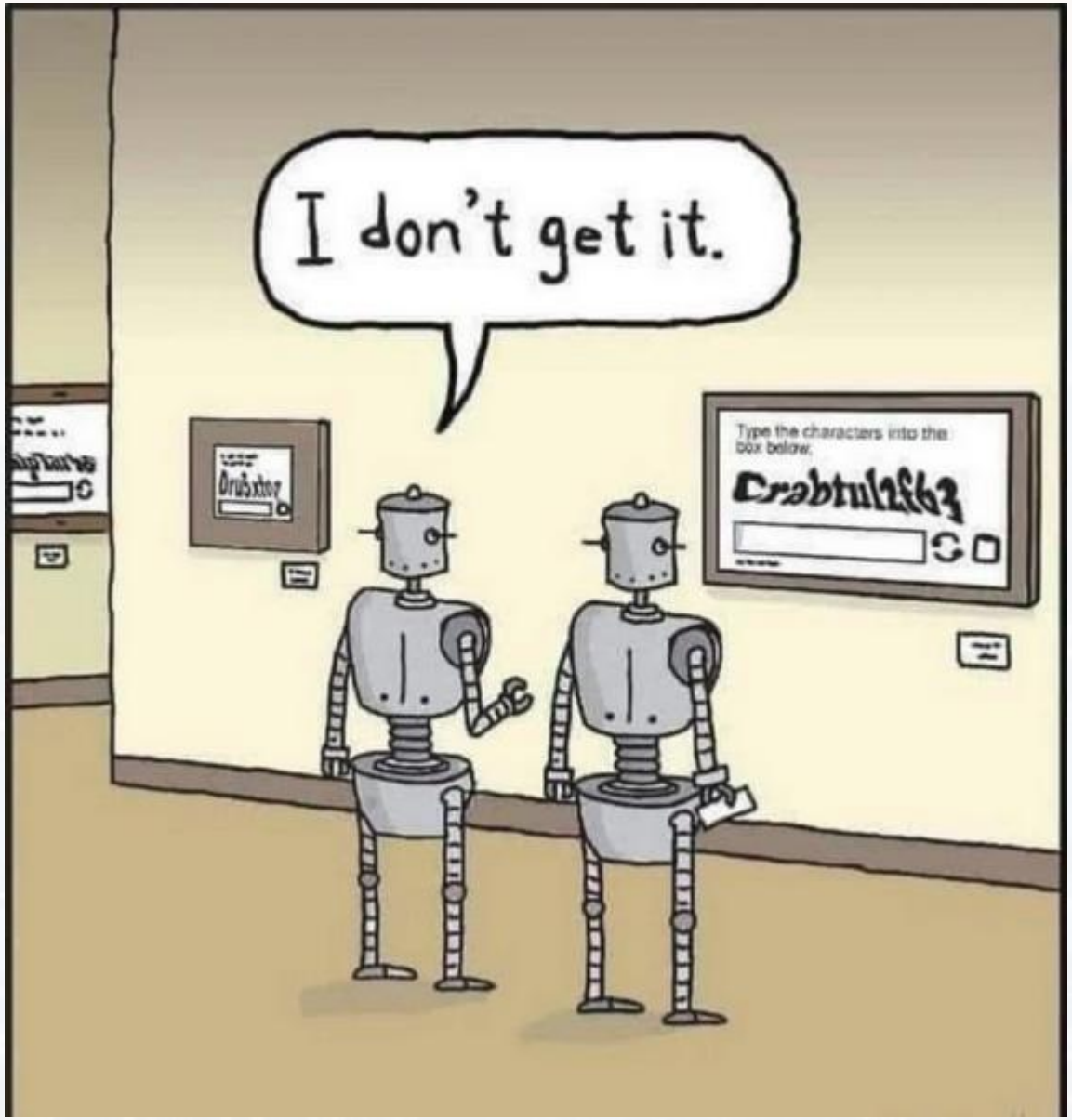
How to split the data?

- 80% training **(60% actual training 20% evaluation)**
- 20% test

evaluation while training!



Clustering and Classification - Supervised



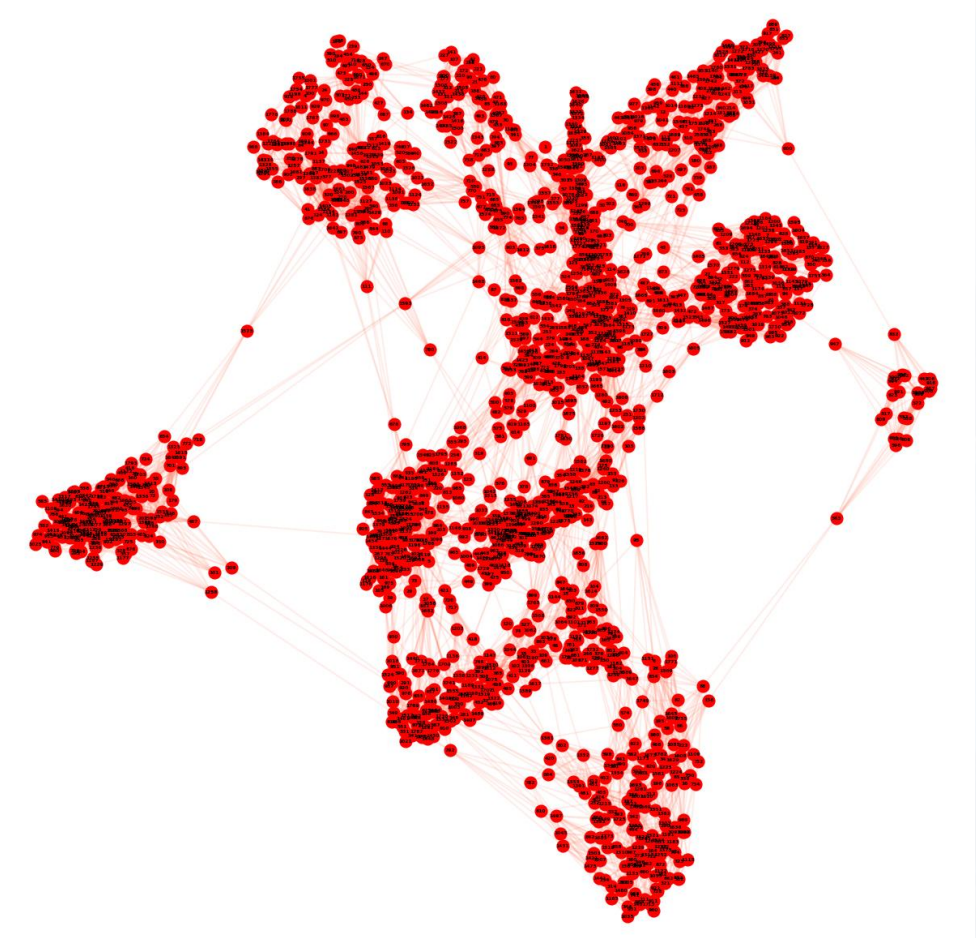
Outline

- Overview
- **Examples**
  - k-NN
  - Naïve Bayes
  - SVM

Clustering and Classification – Supervised

k-**N**earest **N**eighbors: We used that method already implicitly when we:

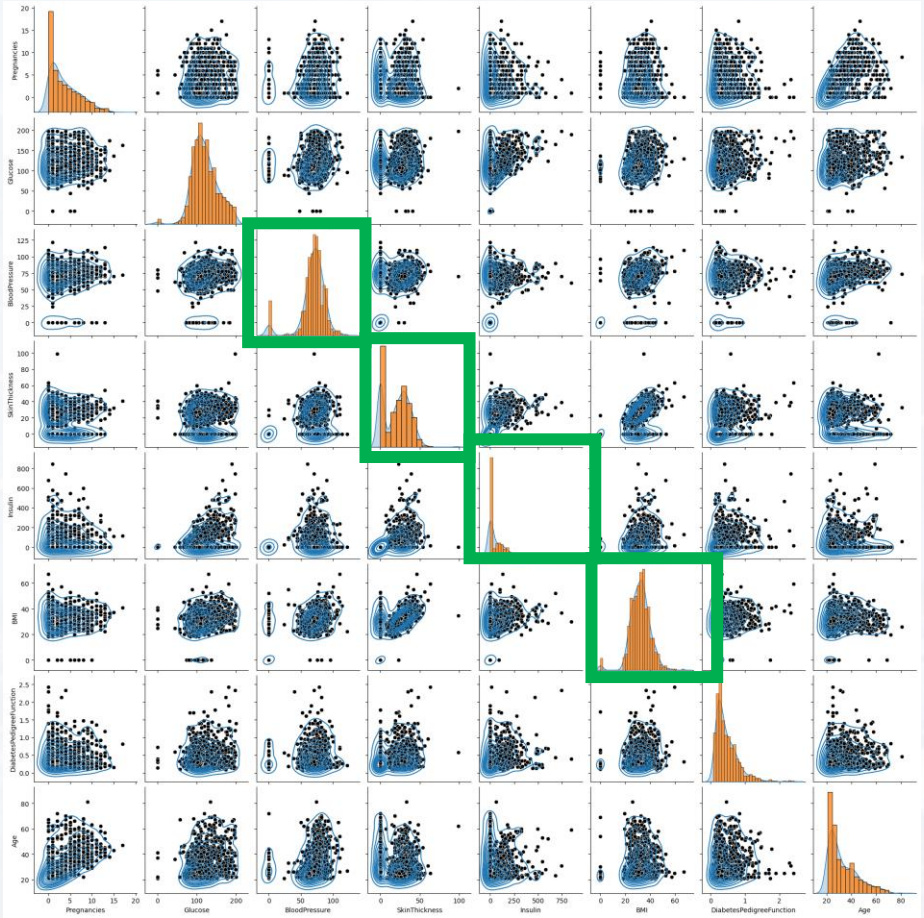
- visualized high-dimensional data (**Module 8**): for node **classification**

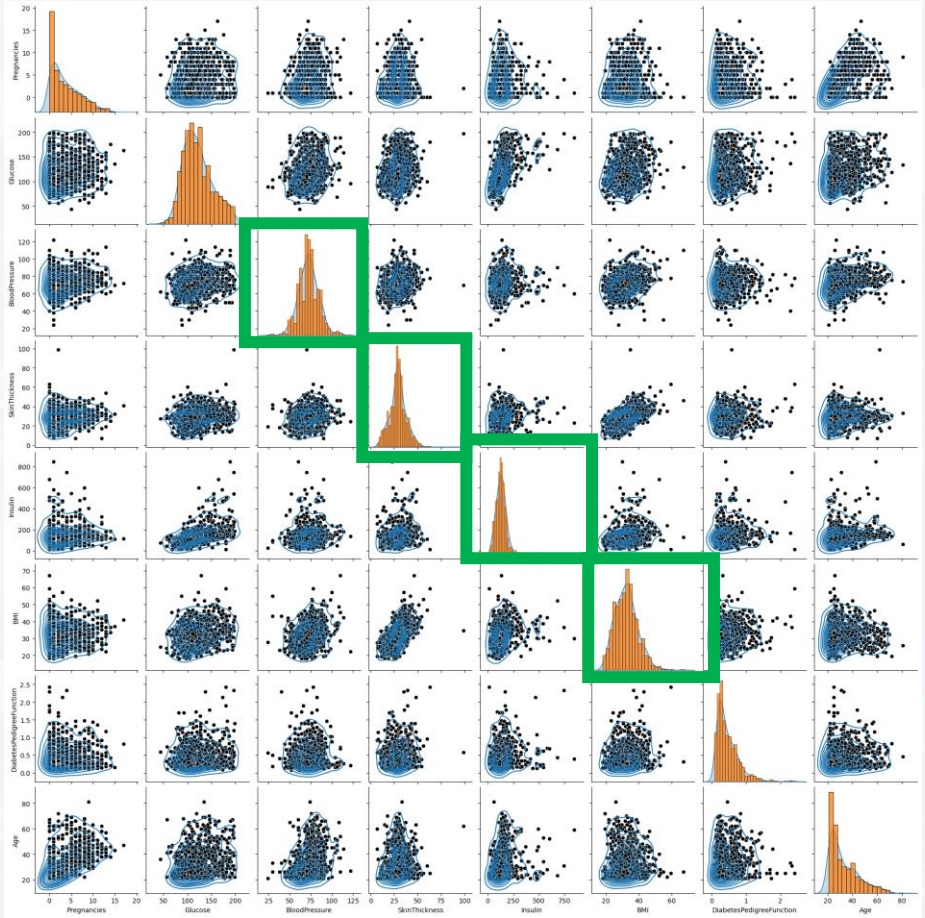


Clustering and Classification – Supervised

k-**N**earest **N**eighbors: We used that method already implicitly when we:

- interpolated missing data (**Module 6**): **regression**





Clustering and Classification – Supervised

idea: 1) determine the **mode** of the categories of the *k* nearest neighbors
2) assign new data point to this category

for classification

$$\hat{Y}(x) = mode\{Y_i\}_{x_i \in N_k(x)}$$

take data point *x* and the category Y_i of its *k* neighbors x_i (*i = 1, 2, ..., k*)

determine the *mode* (most frequent value) of the categories Y_i

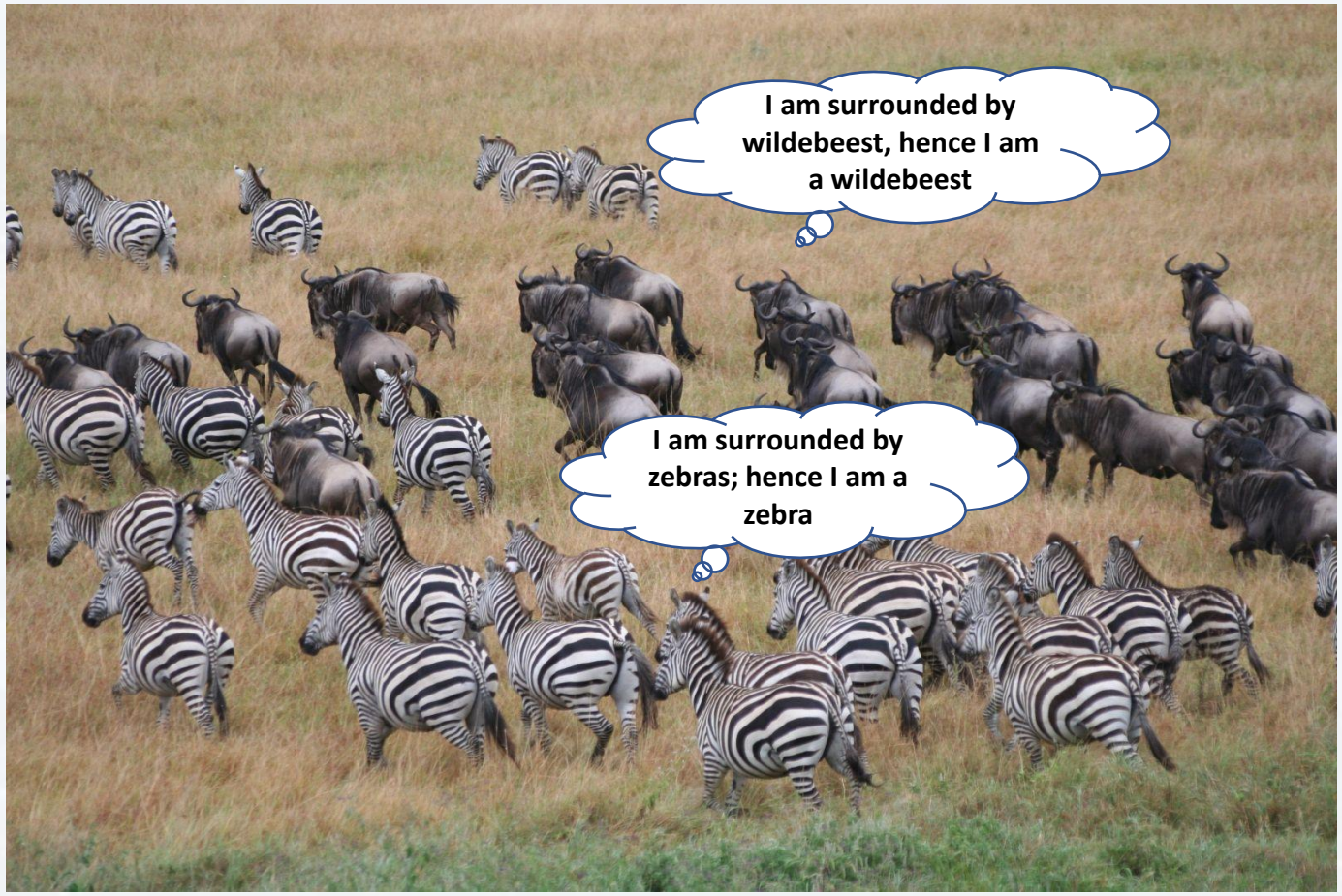
assign the mode of $\{Y_i\}$ to *x*

Clustering and Classification – Supervised

idea: 1) determine the **mode** of the categories of the *k* nearest neighbors
2) assign new data point to this category

for classification

$$\hat{Y}(x) = mode\{Y_i\}_{x_i \in N_k(x)}$$



Clustering and Classification – Supervised

idea: 1) determine the **mode** of the categories of the *k* nearest neighbors
2) assign new data point to this category

for classification

$$\hat{Y}(x) = mode\{Y_i\}_{x_i \in N_k(x)}$$

neighborhood matters!

Clustering and Classification – Supervised

idea: 1) determine the **mode** of the categories of the *k* nearest neighbors
2) assign new data point to this category

for classification

$$\hat{Y}(x) = mode\{Y_i\}_{x_i \in N_k(x)}$$

k matters!

Clustering and Classification – Supervised

idea: 1) determine the **mode** of the categories of the *k* nearest neighbors
2) assign new data point to this category

for classification

$$\hat{Y}(x) = mode\{Y_i\}_{x_i \in N_k(x)}$$

k matters!

Clustering and Classification – Supervised


```
import matplotlib.pyplot as plt
import numpy as np
```

our standard libraries

```
from pyclustering.utils.metric import *
```

for having different distances available

```
from sklearn.neighbors import KNeighborsClassifier
```

performing knearest classification

```
from sklearn import datasets
```

calling the famous “iris” data set

```
Y   = iris.target
X   = iris.data
```

extracting the data as before

```
k   = 5

out = KNeighborsClassifier(k, \
                           metric = distance_metric(type_metric.EUCLIDEAN))
out.fit(X, Y)
```

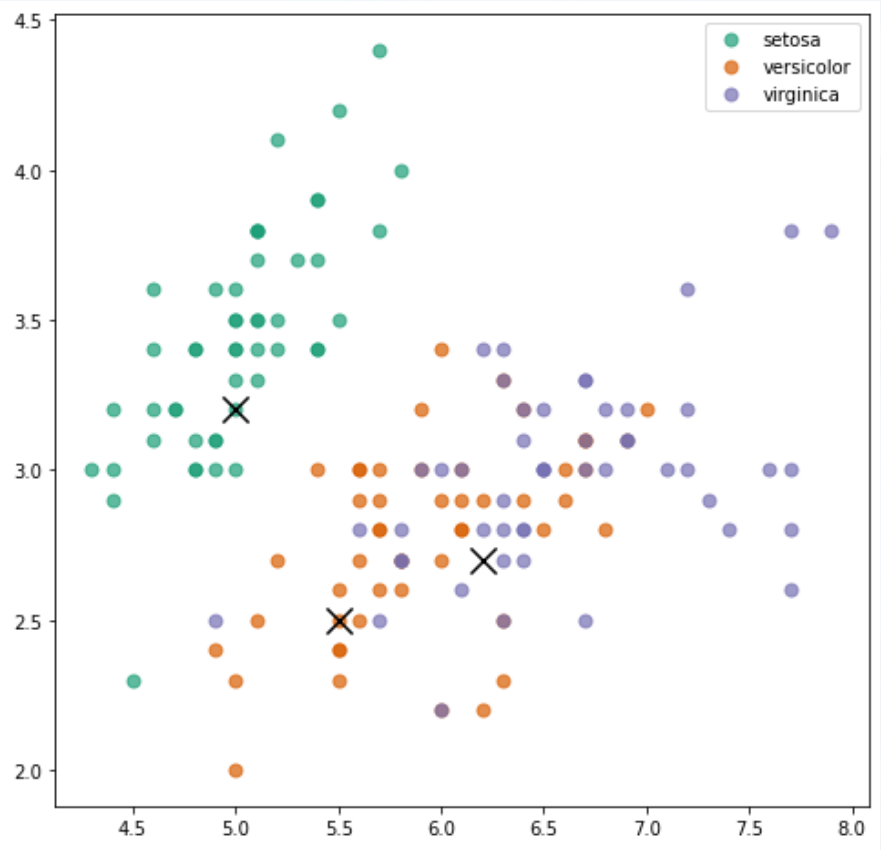
Clustering and Classification – Supervised


```
k    = 5

out = KNeighborsClassifier(k, \
                           metric = distance_metric(type_metric.EUCLIDEAN))
out.fit(X, Y)
```

```
Pred      = out.predict(data_new)
Prob      = out.predict_proba(data_new)
print(Prob)
print(Pred)
```

'setosa'	'versicolor'	'virginica'
1.0	0.0	0.0
0.0	1.0	0.0
0.0	0.4	0.6



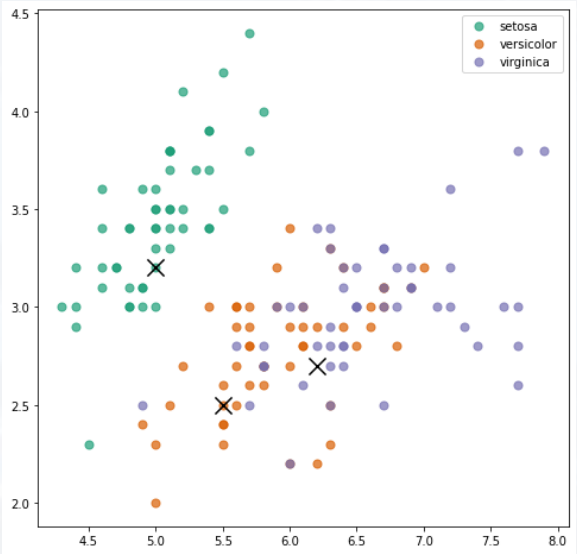
Clustering and Classification – Supervised

summary:

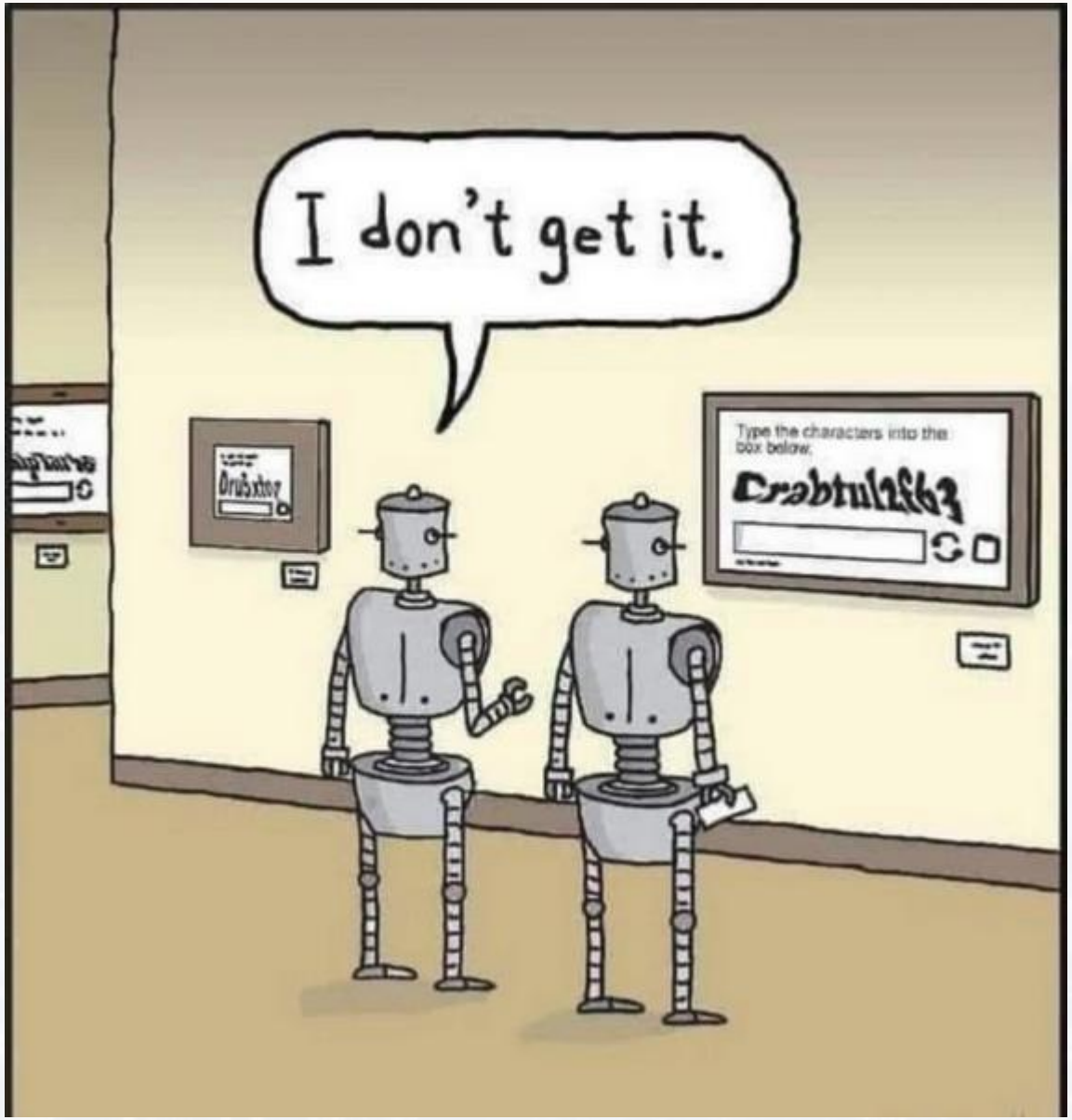
- simple and fast
- supervised
- *k* has to be given and might influence classification
- problems if cluster have unusual shapes

(elongated, hollow inside, scattered)

$$\hat{Y}(x) = mode\{Y_i\}_{x_i \in N_k(x)}$$



Clustering and Classification - Supervised



Outline

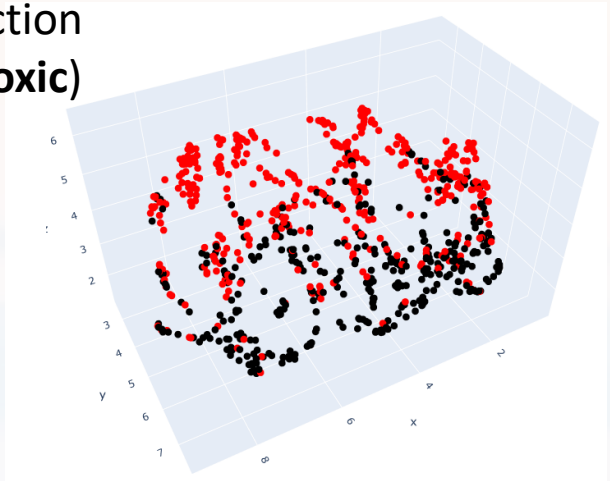
- Overview
- **Examples**
  - k-NN
  - Naïve Bayes
  - SVM

Clustering and Classification – Supervised

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Bayes Theorem

UMAP projection
(**toxic**, **non-toxic**)



$\vec{x}$: vector with all variables (or features)

molecular_weight	electronegativity	bond_lengths	num_hydrogen_bonds	logP	label
357.844435	3.258596	3.181070	4	9.527717	Non-Toxic
436.476397	2.913733	3.617150	9	11.056921	Toxic
350.613984	3.288454	3.156485	5	8.917357	Toxic
361.943582	3.468079	3.277655	7	9.934670	Non-Toxic
366.859512	2.748906	3.174547	3	9.006180	Toxic
308.425424	2.569875	2.994379	7	8.138842	Non-Toxic

K different classes
(here K = 2)

Clustering and Classification – Supervised

$\vec{x}$: vector with all variables (or features)

molecular_weight	electronegativity	bond_lengths	num_hydrogen_bonds	logP	label
357.844435	3.258596	3.181070	4	9.527717	Non-Toxic
436.476397	2.913733	3.617150	9	11.056921	Toxic
350.613984	3.288454	3.156485	5	8.917357	Toxic
361.943582	3.468079	3.277655	7	9.934670	Non-Toxic
366.859512	2.748906	3.174547	3	9.006180	Toxic
308.425424	2.569875	2.994379	7	8.138842	Non-Toxic

K different classes (here K = 2)

$\boldsymbol{P(C_k|\vec{x})}$: **probability that datapoint belongs to class C_k, given $\vec{x}$**

$\boldsymbol{P(\vec{x}|C_k)}$: **probability that datapoint has features $\vec{x}$, given class C_k**

$$\boldsymbol{P(A|B)} = \frac{P(B|A)\boldsymbol{P(A)}}{\boldsymbol{P(B)}}$$

Bayes Theorem

$$P(C_k|\vec{x}) = \frac{\boxed{P(\vec{x}|C_k)}P(C_k)}{P(\vec{x})} = \frac{P(C_k)}{P(\vec{x})}\boxed{\prod_{i=1}^{I} P(x_i|C_k)} \sim P(C_k)\prod_{i=1}^{I} P(x_i|C_k)$$

$$\sum_{k=1}^{K} P(C_k|\vec{x}) = 1$$

Naïve Bayes:

- all features are **mutually independent**
- i. e.: no **correlation** between features
- features can be factorized

Clustering and Classification – Supervised

$$P(C_k|\vec{x}) = \frac{P(\vec{x}|C_k)P(C_k)}{P(\vec{x})} = \frac{P(C_k)}{P(\vec{x})}\prod_{i=1}^{I} P(x_i|C_k)$$

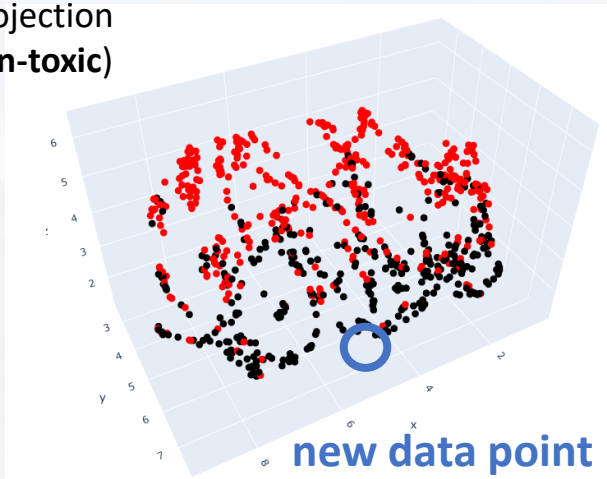
$$\sim P(C_k)\prod_{i=1}^{I} P(x_i|C_k)$$

$$\sum_{k=1}^{K} P(C_k|\vec{x}) = 1$$

$P(C_k|\vec{x})$: probability that datapoint belongs to class C_k, given $\vec{x}$

$P(\vec{x}|C_k)$: probability that datapoint has features $\vec{x}$, given class C_k

UMAP projection
(**toxic**, **non-toxic**)



new data point: finding the class C_k, that maximizes $P(C_k|\vec{x})$

$$k_{new} = \underset{k}{argmax}\left\{P(C_k)\prod_{i=1}^{I} \boxed{P(x_i|C_k)}\right\}$$

from the training data → supervised learning

different models for $P(x_i|C_k)$

- Multinomial (x_i is discrete)
- Gaussian (x_i is normally dist)
- …

Python:

```
from sklearn.naive_bayes import *
from sklearn.preprocessing import MinMaxScaler
```

importing methods for naïve bayes

scaling/normalizing data

```
Train = pd.read_csv('molecular_train_gbc_cat.csv')
Test  = pd.read_csv('molecular_test_gbc_cat.csv')

XTrain = Train.drop('label', axis = 1).values
YTrain = Train['label']

XTest = Test.drop('label', axis = 1).values
YTest = Test['label']
```

```
print(YTrain[:10])
```

```
0        Toxic
1        Toxic
2    Non-Toxic
3    Non-Toxic
4    Non-Toxic
5        Toxic
6    Non-Toxic
7    Non-Toxic
8        Toxic
9    Non-Toxic
Name: label, dtype: object
```

```
scaler   = MinMaxScaler(feature_range = (0, 1))
XTrainS  = scaler.fit_transform(XTrain)
```

scaling the data to mean = 0 and std = 1

```
gnb = GaussianNB()
Fit = gnb.fit(XTrainS, YTrain)
```

the actual fit

applying the model to the test data set

```
XTestS  = scaler.transform(XTest)
```

scaling the test set **without** fitting

```
Ypred   = Fit.predict(XTestS)
Probs   = Fit.predict_proba(XTestS)
```

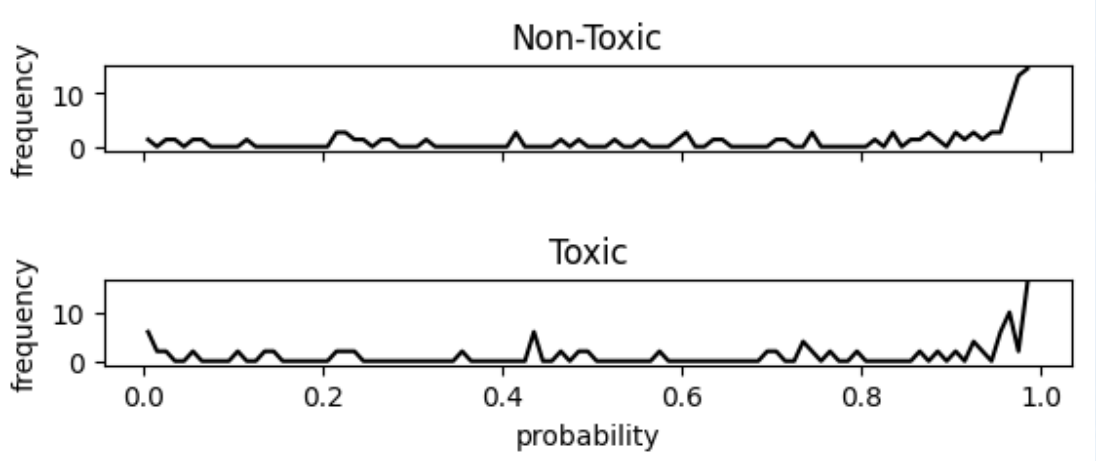
predicting the class C_k and calculating the probabilities

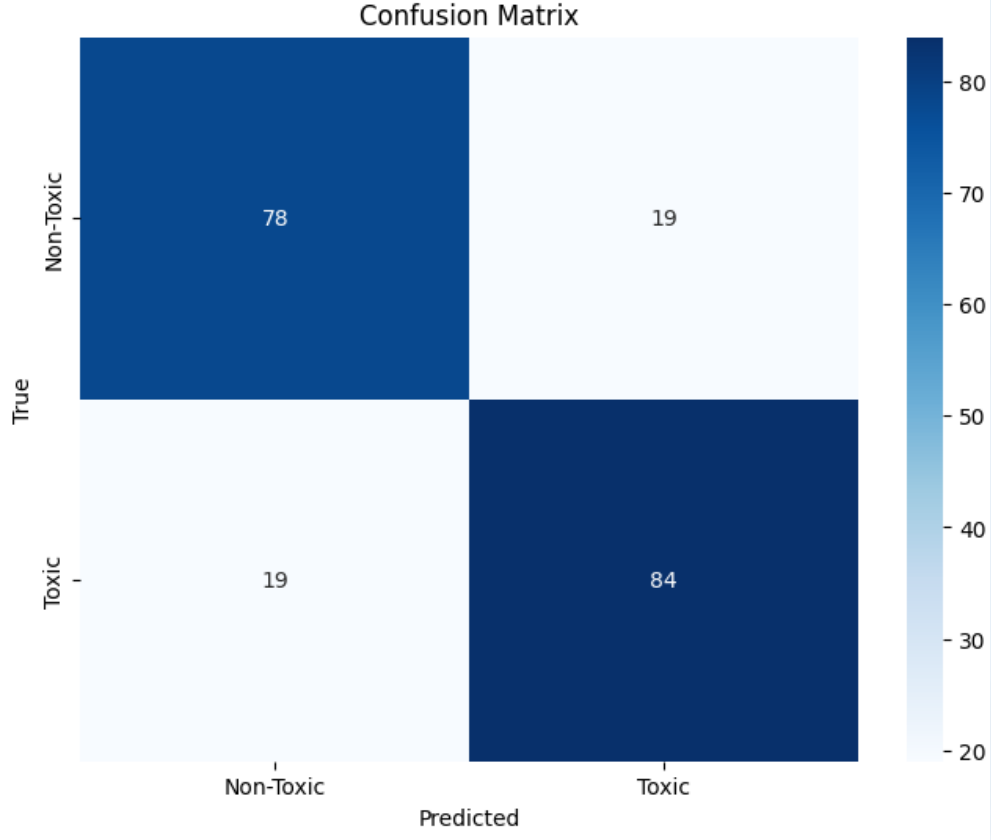
Clustering and Classification – Supervised


```
XTestS  = scaler.transform(XTest)

Ypred   = Fit.predict(XTestS)
Probs   = Fit.predict_proba(XTestS)
```

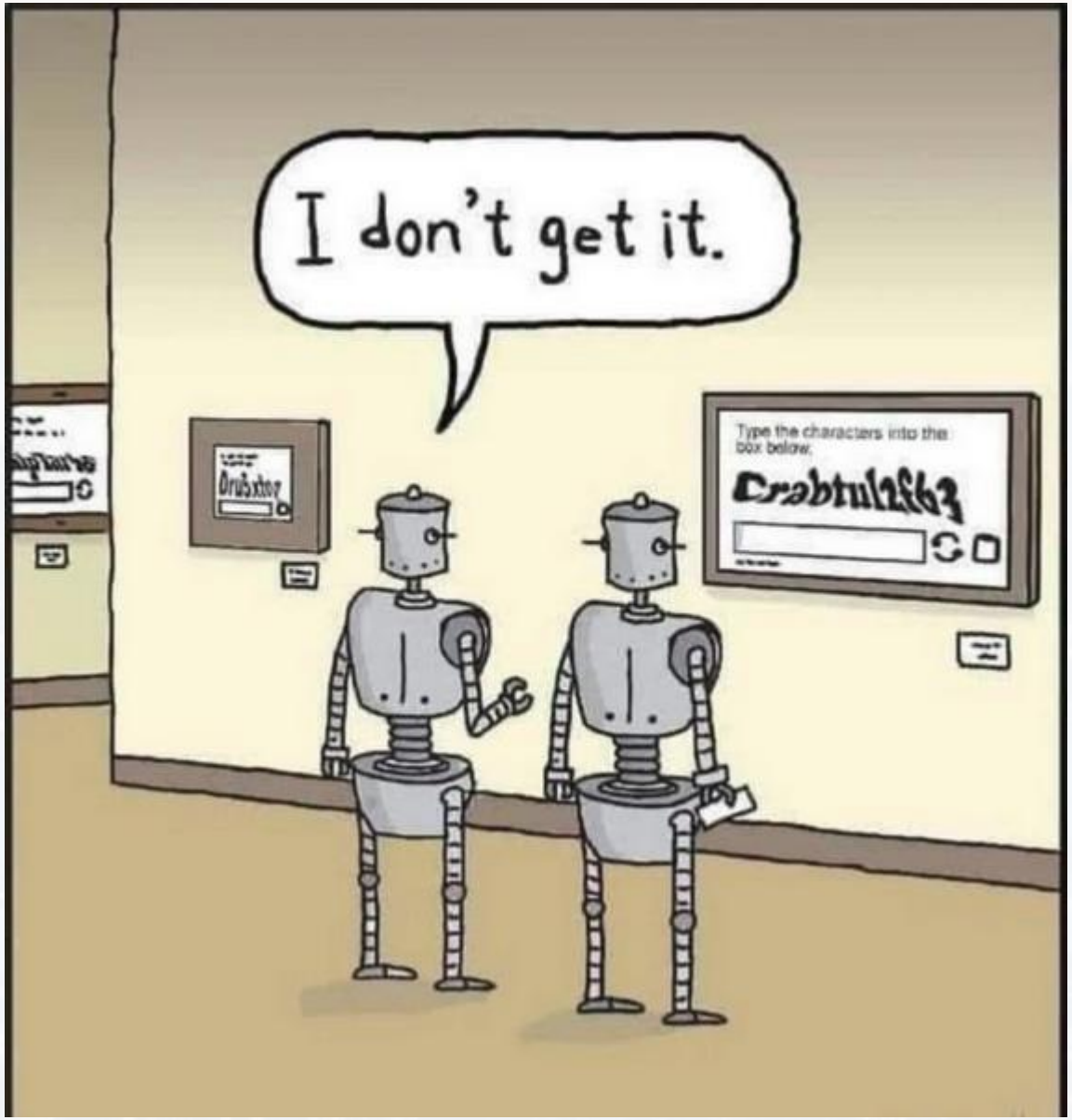
evaluation





see
Walk_Through_NaiveBayes.ipynb

Clustering and Classification - Supervised



Outline

- Overview
- **Examples**
  - k-NN
  - Naïve Bayes
  - SVM

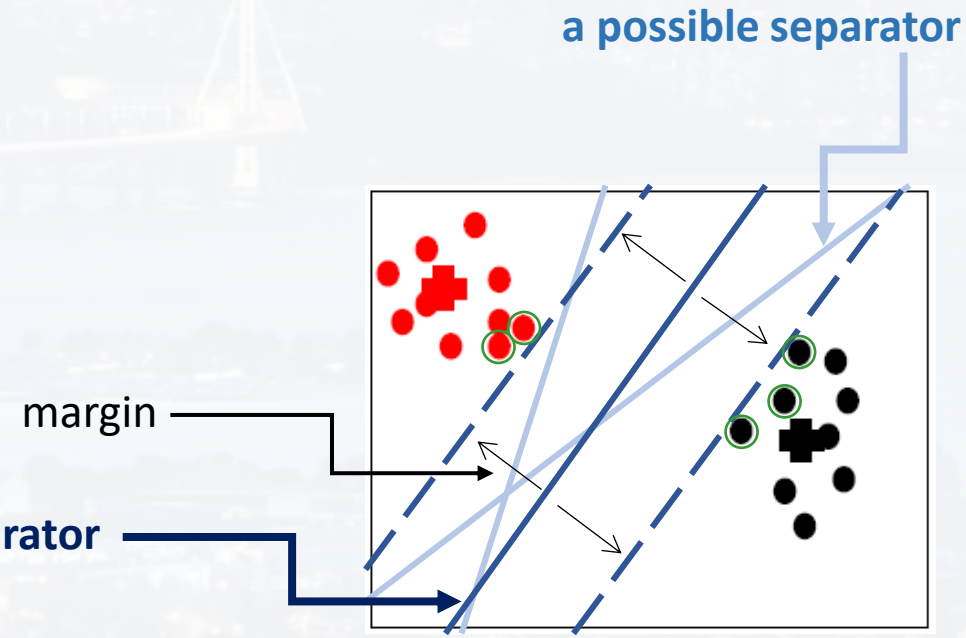
Clustering and Classification – Supervised

SVM = **S**upport **V**ector **M**achine

idea:

1) finds best **linear** classifier for separating **two** classes by **maximizing margin** using **support vectors**
2) assign new data points to these categories
3) **supervised** learning
4) uses the "kernel trick"

○ support vectors (data points at the edge)



1) What if linear separation is not possible?
2) Is there a multiclass SVM?

Clustering and Classification – Supervised

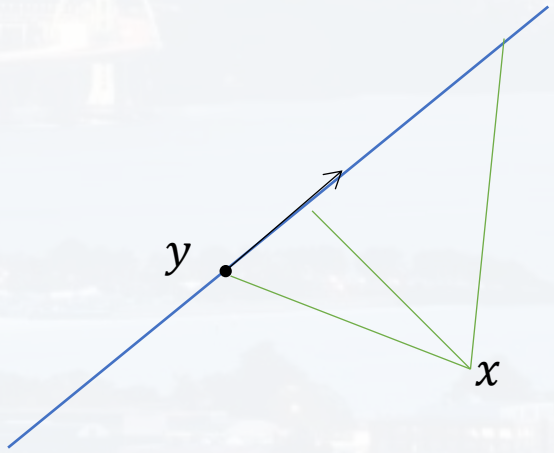
1) **What if linear separation is not possible?**
2) Is there a multiclass SVM?

distance $d(x, y)$ between two points x and y

dot product $\langle\cdot\rangle$ of the vector $x - y$ with itself: $d^2(x, y) = \langle x - y, x - y\rangle$

$$= \langle x, x\rangle - 2\langle x, y\rangle + \langle y, y\rangle$$

$$= |x - y|$$



Clustering and Classification – Supervised

1) **What if linear separation is not possible?**
2) Is there a multiclass SVM?

distance $d(x, y)$ between two points x and y

dot product $\langle\cdot\rangle$ of the vector $x - y$ with itself: $d^2(x, y) = \langle x - y, x - y\rangle$

$$= \langle x, x\rangle - 2\langle x, y\rangle + \langle y, y\rangle$$

$$= |x - y|$$

definition: kernel $\mathrm{K}(x, y) := \langle \phi(x), \phi(y)\rangle$ example: linear kernel $\phi: \mathrm{x} \mapsto \mathrm{x}$ $\phi(x) = x$

$$d_\phi^2(x, y) = \langle x, x\rangle - 2\langle x, y\rangle + \langle y, y\rangle = x^2 - 2xy + y^2 = (x - y)^2$$

$$d_\phi^2(x, y) = |x - y|$$

the linear separator, we know already

Clustering and Classification – Supervised

1) **What if linear separation is not possible?**
2) Is there a multiclass SVM?

distance $d(x, y)$ between two points x and y

dot product $\langle\cdot\rangle$ of the vector $x - y$ with itself: $d^2(x, y) = \langle x - y, x - y\rangle$

$$= \langle x, x\rangle - 2\langle x, y\rangle + \langle y, y\rangle$$

$$= |x - y|$$

definition: kernel $\mathrm{K}(x, y) := \langle \phi(x), \phi(y)\rangle$

example: RBF (**r**adial **b**asis **f**unction)

$$K(x, y) = exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

$$d_\phi^2\big(\phi(x), \phi(y)\big) = 2\left[1 - exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)\right]$$

Clustering and Classification – Supervised

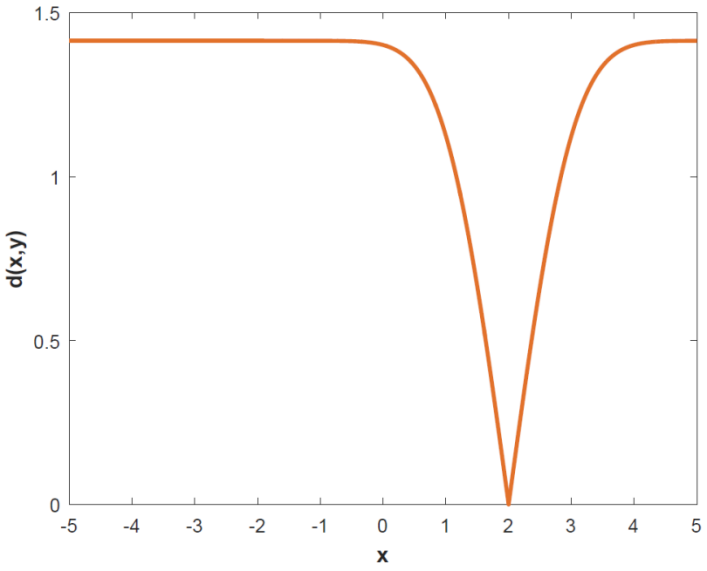
1) **What if linear separation is not possible?**
2) Is there a multiclass SVM?

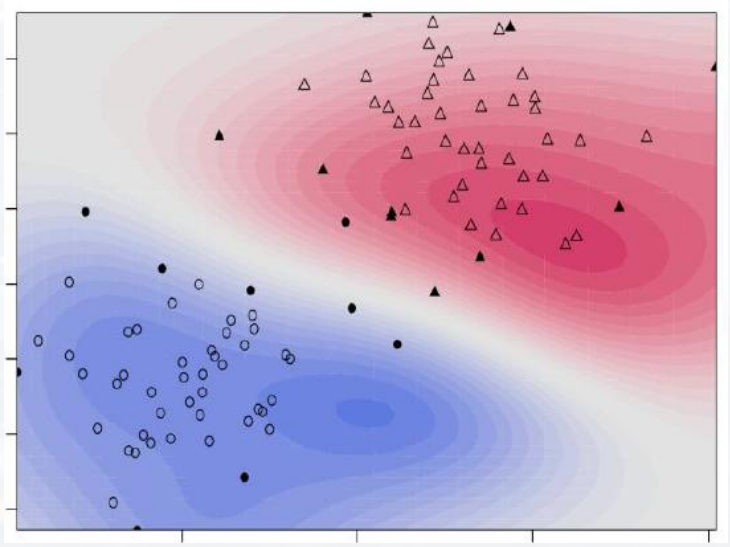
definition: kernel $\mathrm{K}(x,y) := \langle \phi(x), \phi(y) \rangle$

example: RBF (**r**adial **b**asis **f**unction)

$$K(x,y) = exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

$$d_\phi^2(\phi(x),\phi(y)) = 2\left[1 - exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)\right]$$





Clustering and Classification – Supervised

1) **What if linear separation is not possible?**
2) Is there a multiclass SVM?

kernel trick: - introducing a non-linear kernel via distances → non linear separator!

<u>kernels available in `sklearn`:</u>

- linear:

$$K(x,y) = \|x - y\|$$

- Gaussian aka RBF (**r**adial **b**asis **f**unction):

$$K(x,y) = exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

in `sklearn` we can adjust $\boldsymbol{\gamma} := \frac{\mathbf{1}}{\mathbf{2}\boldsymbol{\sigma}^{\mathbf{2}}}$

- polynomial:

$$K(x,y) = \sum_{n=1}^{N} \|x-y\|^n$$

in `sklearn` we can adjust $\boldsymbol{N}$

- sigmoidal:

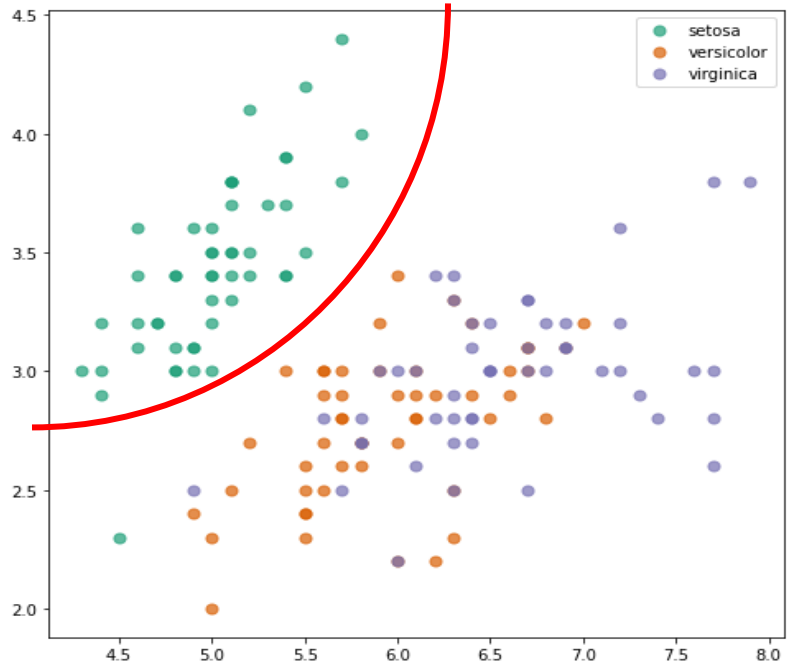
$$K(x,y) = \frac{e^{\|x-y\|}}{1 + e^{\|x-y\|}}$$

Clustering and Classification – Supervised

1) What if linear separation is not possible?
2) **Is there a multiclass SVM?**

→ one vs rest / one vs one

green vs the rest
→ storing probabilities

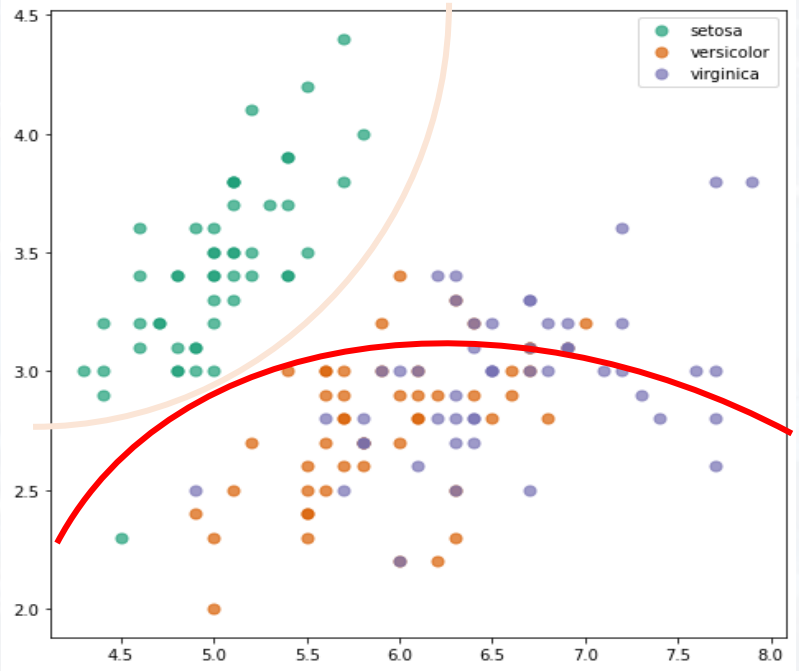


Clustering and Classification – Supervised

1) What if linear separation is not possible?
2) **Is there a multiclass SVM?**

→ one vs rest / one vs one

green vs the rest
→ storing probabilities



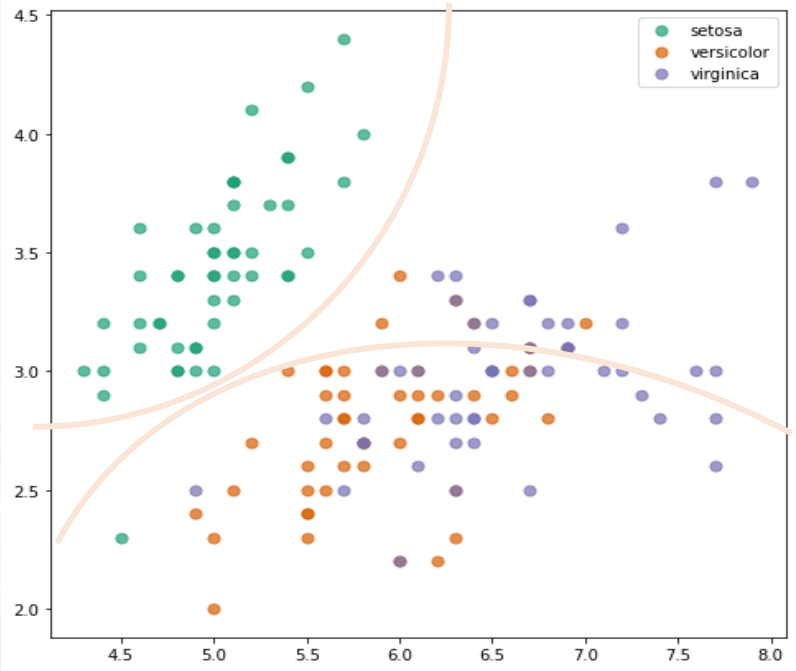
orange vs the rest
→ storing probabilities

Clustering and Classification – Supervised

1) What if linear separation is not possible?
2) **Is there a multiclass SVM?**

→ one vs rest / one vs one

green vs the rest
→ storing probabilities



blue vs the rest
→ storing probabilities

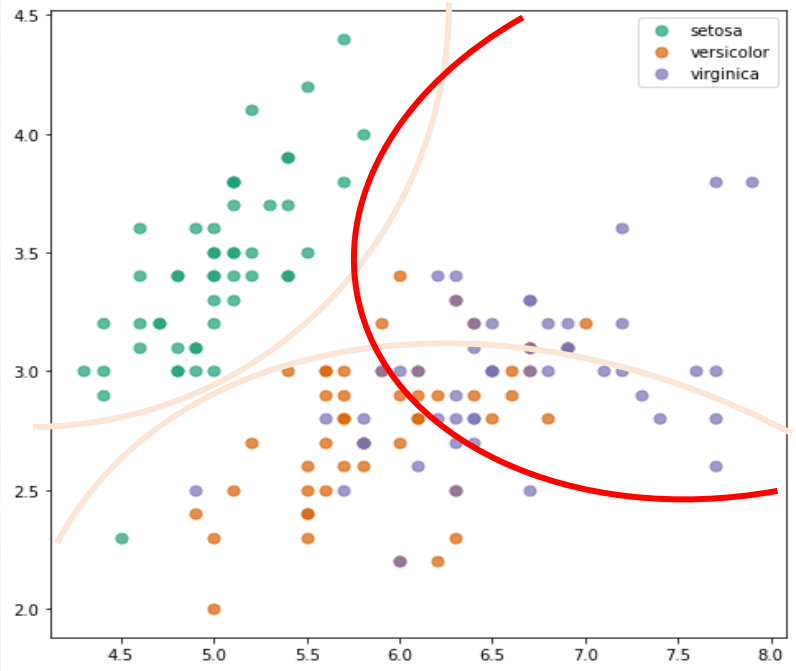
orange vs the rest
→ storing probabilities

Clustering and Classification – Supervised

1) What if linear separation is not possible?
2) **Is there a multiclass SVM?**

→ one vs rest / one vs one

green vs the rest
→ storing probabilities



blue vs the rest
→ storing probabilities

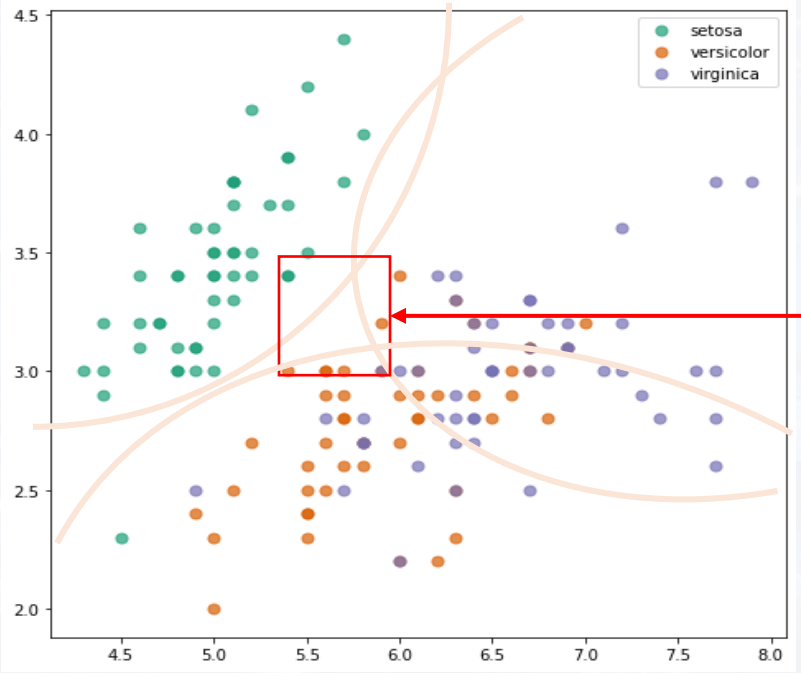
orange vs the rest
→ storing probabilities

Clustering and Classification – Supervised

1) What if linear separation is not possible?
2) **Is there a multiclass SVM?**

→ one vs rest / one vs one

green vs the rest
→ storing probabilities



three probabilities for each data point
→ assign class to most probable value

k class classification with two-class discriminant functions is ambiguous!

Clustering and Classification – Supervised

see **Walk_Through_SVM.ipynb**

```
from sklearn import svm
```

1) setting up the model & 2) fitting the model

running analysis with different kernel

```
outlinear = svm.SVC(kernel = 'linear', C = 1, decision_function_shape = 'ovr')
linear    = outlinear.fit(X2D, Y)
```

one **v**ersus **r**est

L2 regularization parameter for error tolerance when calculating the classifier

```
outrbf    = svm.SVC(kernel = 'rbf', gamma = 1, C = 1, \
                                    decision_function_shape = 'ovr')
rbf       = outrbf.fit(X2D, Y)
```

$$\gamma := \frac{1}{2\sigma^2}$$

```
outpoly   = svm.SVC(kernel = 'poly', degree = 3, C = 1,\
                                    decision_function_shape = 'ovr')
poly      = outpoly.fit(X2D, Y)
```

refers to N in $\sum_{n=1}^{N} \|x - y\|^n$

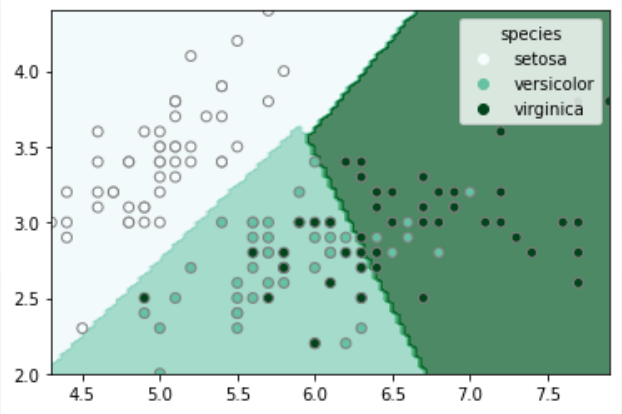
```
outsig    = svm.SVC(kernel = 'sigmoid', C = 1, decision_function_shape = 'ovr')
sig       = outsig.fit(X2D, Y)
```

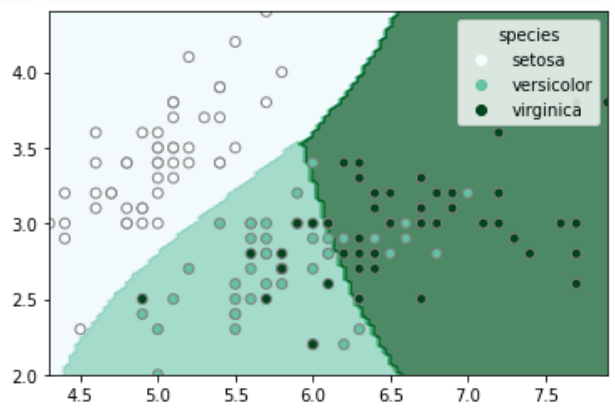
Clustering and Classification – Supervised


```
from sklearn import svm
```

see **Walk_Through_SVM.ipynb**

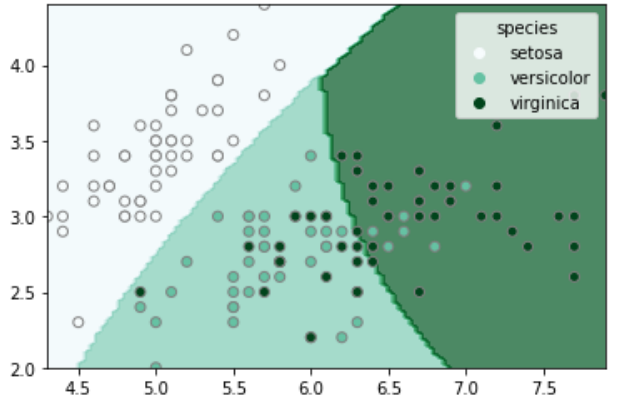
$$K(x,y) = \|x-y\|$$

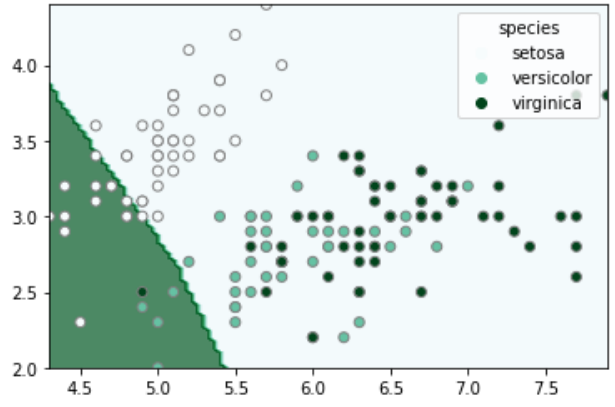




$$K(x,y) = exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$$

$$K(x,y) = \sum_{n=1}^{N} \|x-y\|^n$$





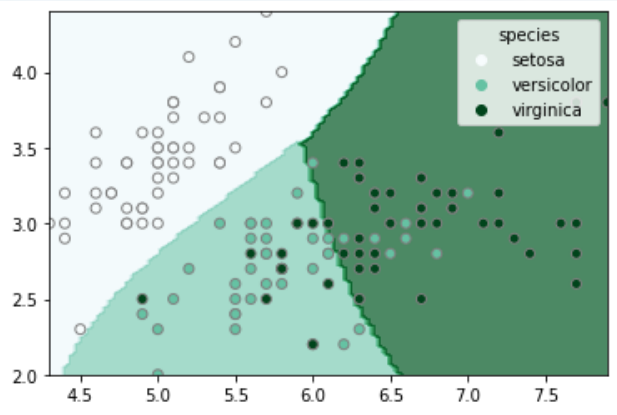
$$K(x,y) = \frac{e^{\|x-y\|}}{1+e^{\|x-y\|}}$$

Clustering and Classification – Supervised

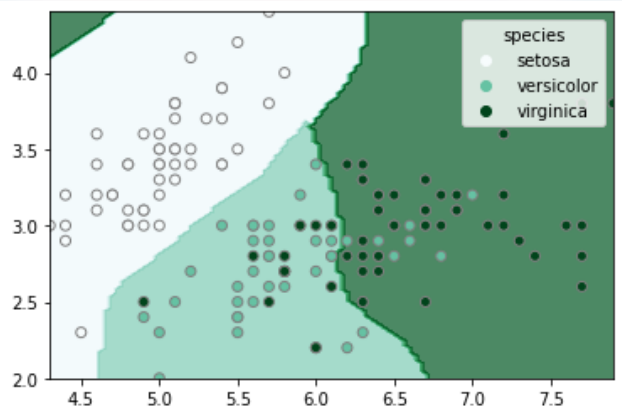

```
from sklearn import svm
```

see **Walk_Through_SVM.ipynb**

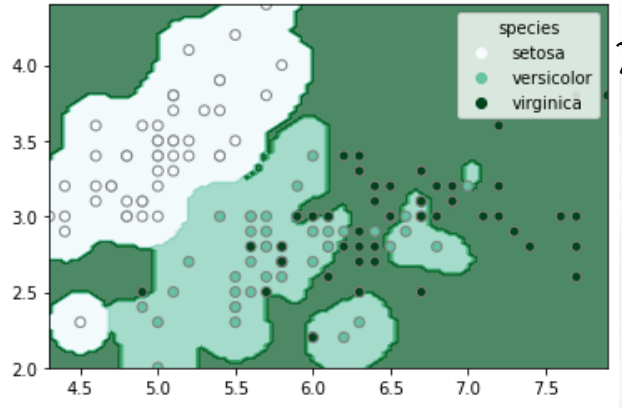
$$K(x,y) = exp\left(-\frac{1}{2\sigma^2}\|x-y\|^2\right)$$



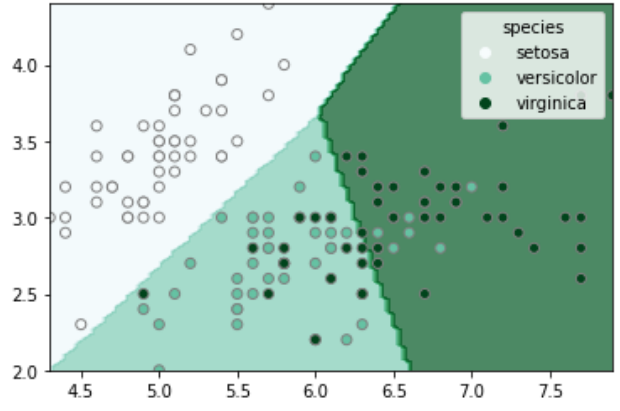
$$\gamma := \frac{1}{2\sigma^2} = 1$$



$$\gamma := \frac{1}{2\sigma^2} = 5$$



$$\gamma := \frac{1}{2\sigma^2} = 50$$



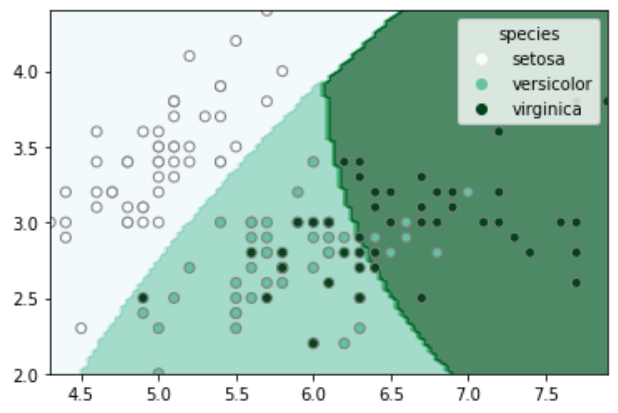
$$\gamma := \frac{1}{2\sigma^2} = 0.1$$

Clustering and Classification – Supervised

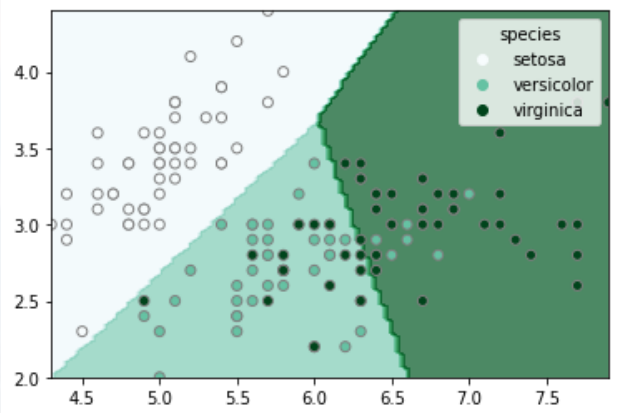

```
from sklearn import svm
```

see **Walk_Through_SVM.ipynb**

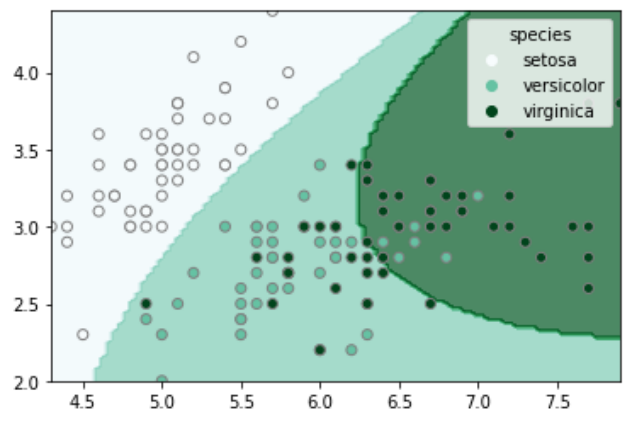
$$K(x, y) = \sum_{n=1}^{N} \|x - y\|^n$$



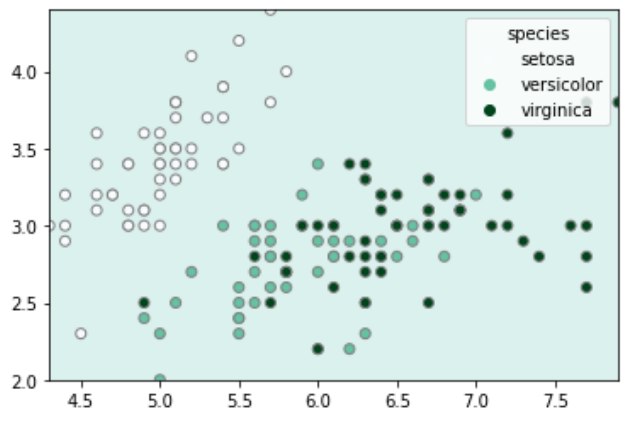
N = 3



N = 1



N = 5



N = 0

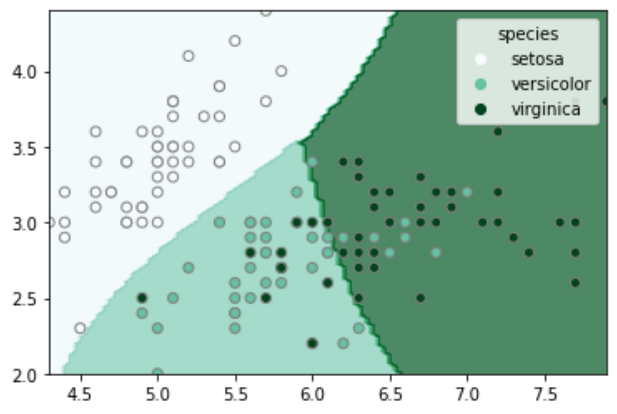
Clustering and Classification – Supervised


```
from sklearn import svm
```

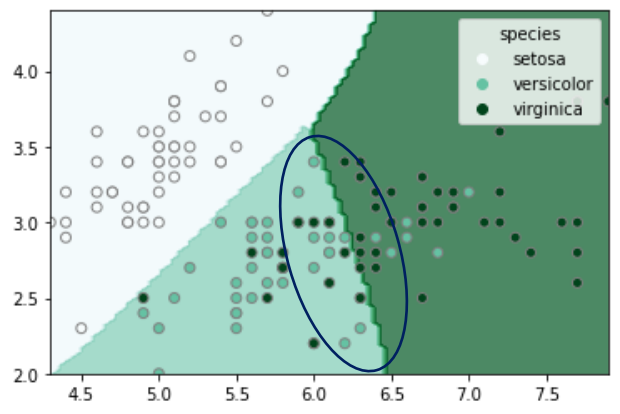
see **Walk_Through_SVM.ipynb**

$$K(x,y) = exp\left(-\frac{1}{2\sigma^2}\|x-y\|^2\right)$$

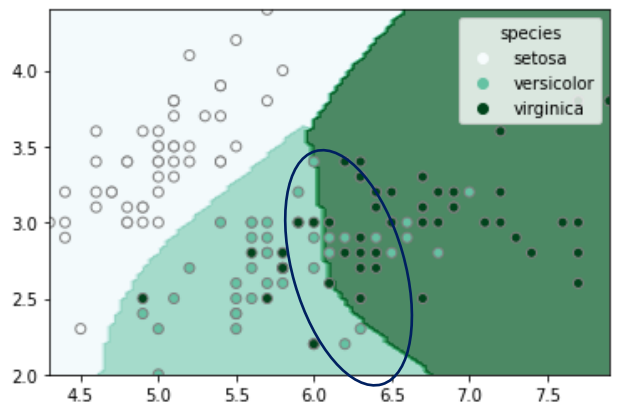
C is a L2 regularization parameter



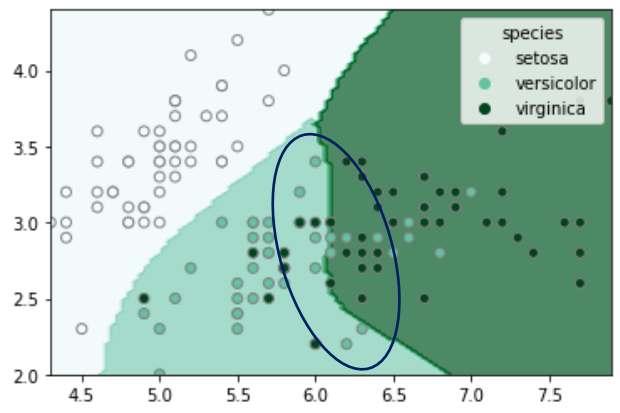
C = 1



C = 0.1



C = 10



C = 20

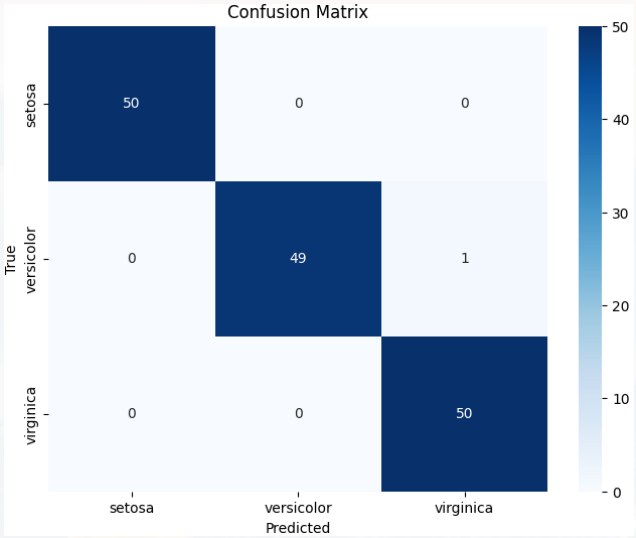
Clustering and Classification – Supervised

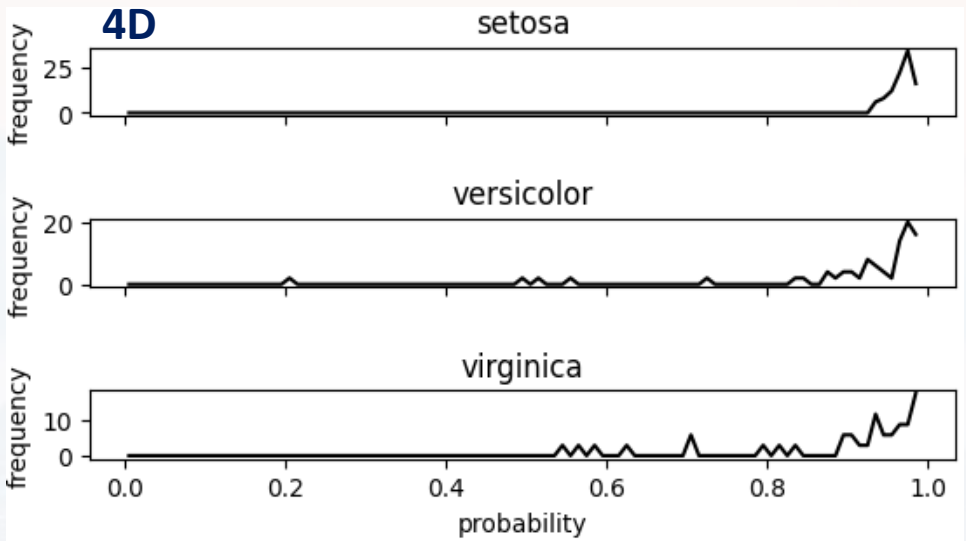
full **4D** data set

3) evaluating the model

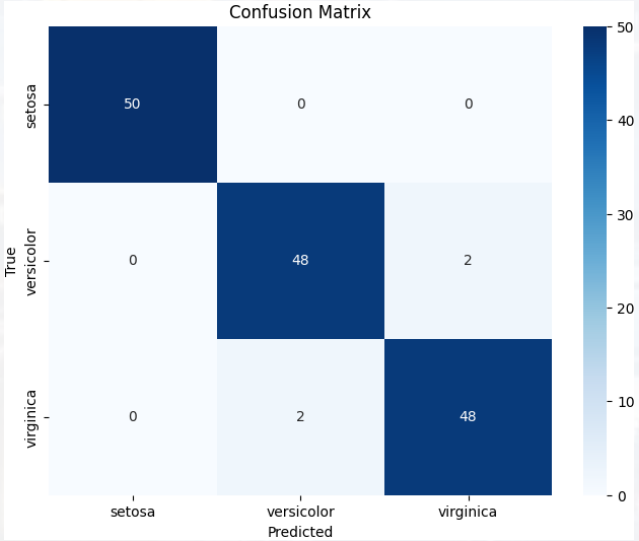
see `Walk_Through_SVM.ipynb`

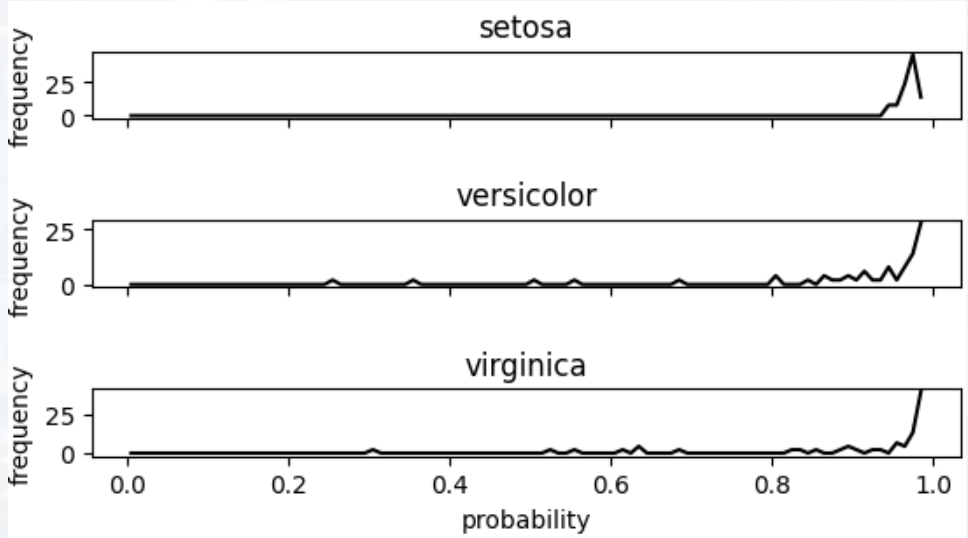
linear
accuracy: 99.3%





Gaussian ($\gamma = 1$)
accuracy: 97.3%





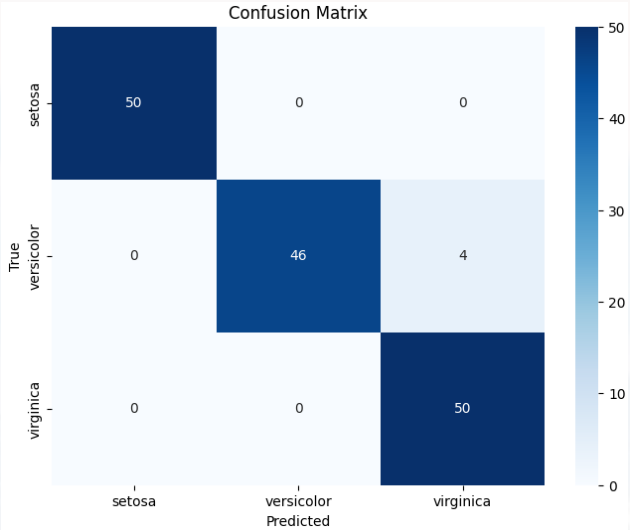
Clustering and Classification – Supervised

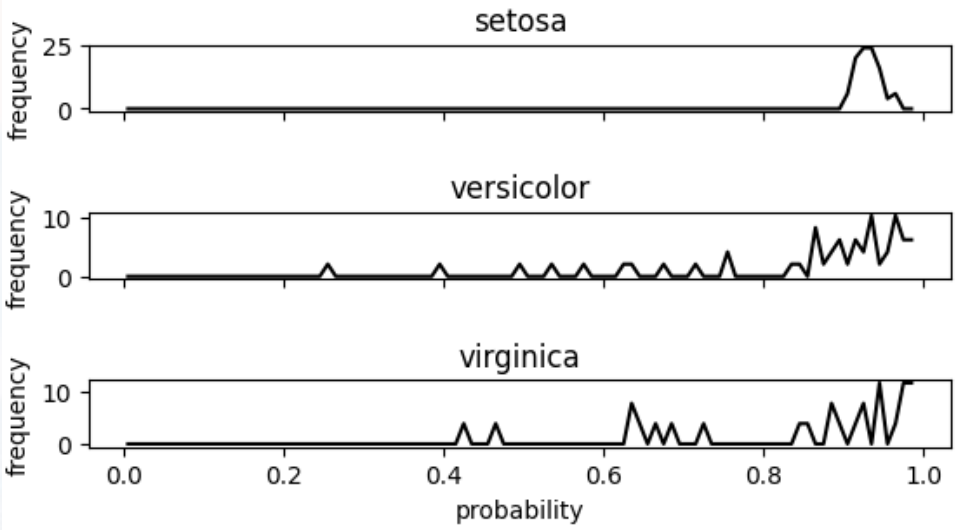
full **4D** data set

polynomial (n = 3)
accuracy: 97.3%

3) evaluating the model

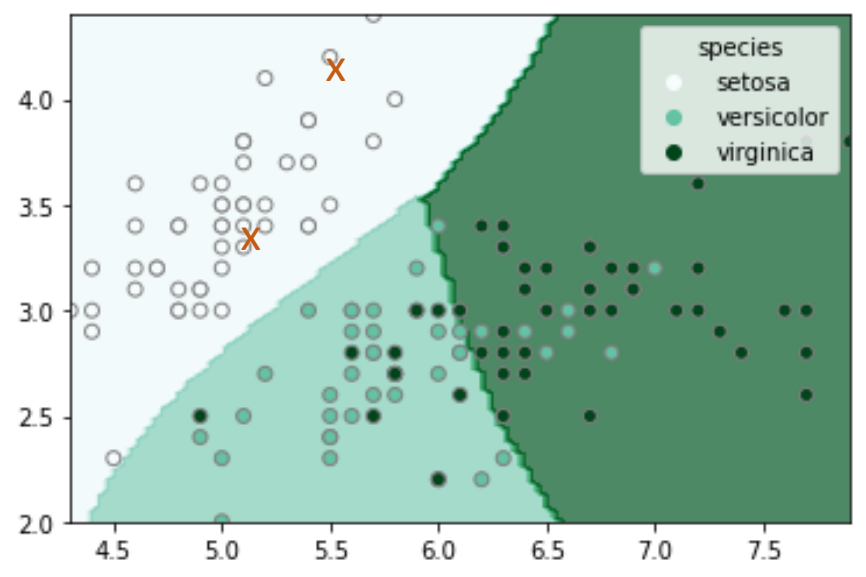
see `Walk_Through_SVM.ipynb`

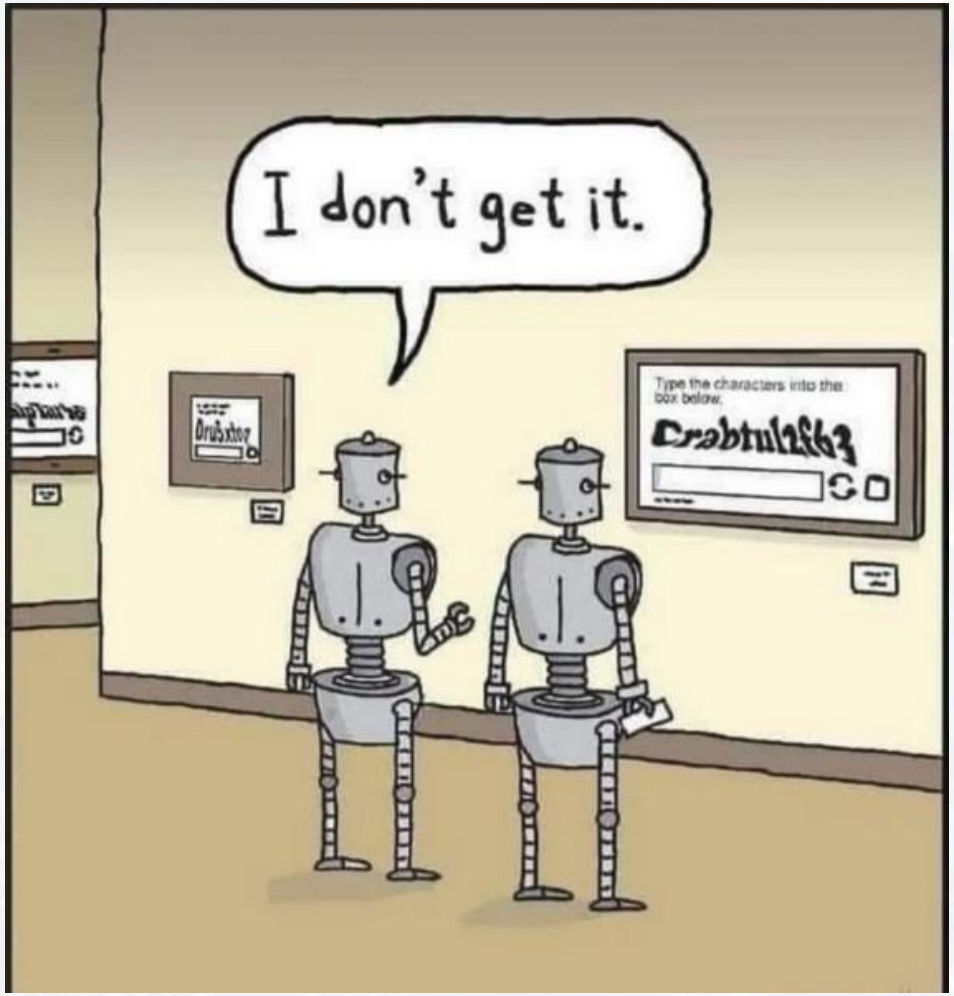




4) applying the model to a new data set

```
ypred = outpoly.predict([[6, 3.5],[6.3, 4.5]])
```



Clustering and Classification - Supervised

Thank you very much for your attention!